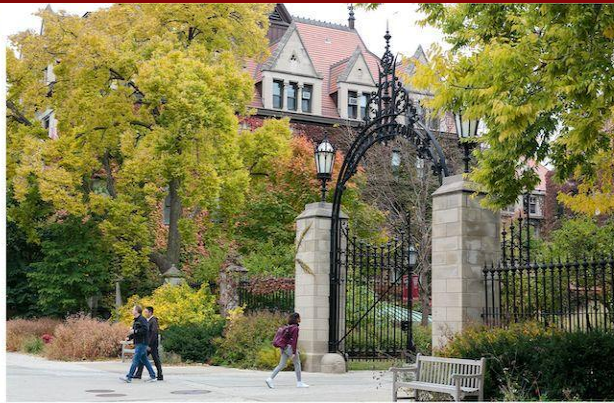


MPCS 52060 - Parallel Programming

M1: Introduction to Parallel Programming



About me

- Jan Hückelheim
- jhueckelheim@uchicago.edu
- Best way to contact me is email or Slack



About me

- B.Sc. and M.Sc. in Computational Engineering Sciences at RWTH Aachen University, Germany



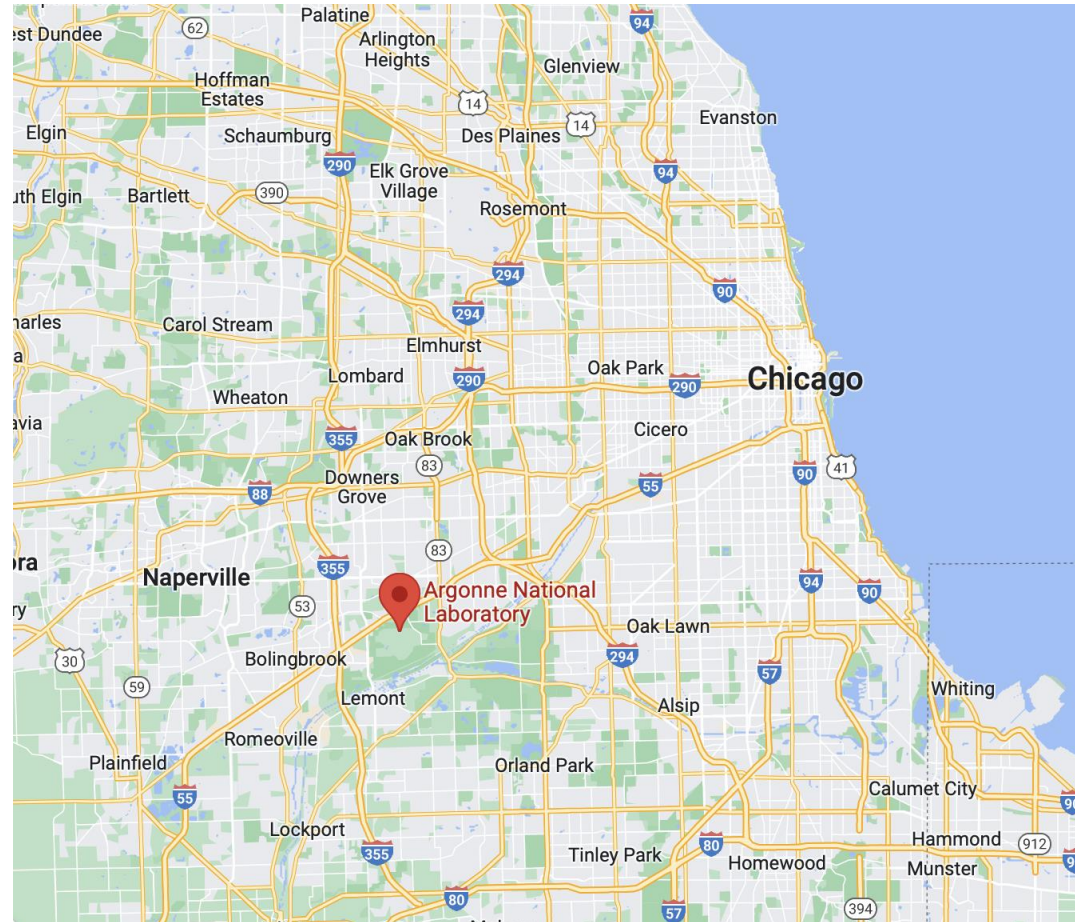
About me

- PhD at University of London 2017
- Postdoc at Imperial College London 2017-2019



About Argonne

- Since 2019 scientific staff at Argonne National Laboratory
- DOE research lab
- Open science: Mostly unclassified, freely available research
- 3400 staff, \$1.2B annual budget
- Internships available every year
- No US citizenship requirement



My Work

- PhD thesis on differentiating through fluid dynamics code running on Intel accelerators (similar to GPUs)
- Current work: verification and automatic differentiation (aka. back-propagation) applied to parallel programs
- >10 years experience in parallel programming
- Mostly Fortran, C/C++, with OpenMP and MPI

My Work

- PhD thesis on differentiating through fluid dynamics code running on Intel accelerators (similar to GPUs)
- Current work: verification and automatic differentiation (aka. back-propagation) applied to parallel programs
- >10 years experience in parallel programming
- Mostly Fortran, C/C++, with OpenMP and MPI
- I'm still learning...

Course Logistics

- Outline of the Syllabus is on Canvas
- There will be 2 assignments and 3 projects
- Final grade:
 - Each assignment is 5%
 - Each project is 20%
 - The midterm is 30%
 - Assignments and Projects on GitHub classroom, invitation links will be shared on Canvas
 - Assignments and projects must be submitted to gradescope

Assignment and Project Policies

- Basic gist: do not submit anyone else's work as your own; do not share your work with anyone else. Plagiarism can have serious consequences.
- Do not submit ChatGPT or other AI-generated content
- If you use code from public sources, add citations and references, clearly indicating what part of the code is recycled. Also, follow copyright rules!
- Detailed rules are on Canvas.

Midterm Policies

- The midterm will be in person, during class time, week 6
- Pen and paper, with a mix of questions and programming
- You may bring a one-page double-sided hand-written cheat sheet
- Programming questions: Small syntax errors or misnamed functions are okay, if the intent is clear.

Getting Help

- Use Slack to ask questions
- We will use Ed for class questions
- Email TA if this does not work, or come to office hours.
- Office hours starting in week 2, just before class
- Contact me at jhueckelheim@uchicago.edu or Slack
- Ask your instructor for clarification when in doubt about course content, policies, etc.

About this course

- The course has been given by Lamont Samuels in past quarters, and the general structure and some slides are created by him (his slides are marked at the bottom).

Motivation for Parallelism:

A real-world analogy

Motivation for Parallelism

Sequential vs Parallel Programming

- Sequential programming:
 - One sequence of program instructions
 - Each instruction can only start after the previous instructions have finished
- Parallel programming:
 - Multiple sequences of instructions run in parallel
 - Within each sequence, instructions can only start after the previous instructions have finished
 - Synchronization instructions may introduce dependencies across sequences

Is Sequential Programming Good Enough?

- Sequential programming is easier. Why do we need more?
- Won't future computers be faster, so I don't need to worry about this?

Transistors and speed

- Increase in single processor performance has been driven by increasing the *density of transistors* on the CPU chip.
 - ▶ **Transistors** - electronic components on integrated circuits that act as switches in order to construct logical gates. We use logic gates to form logical units capable of arithmetic and complex logical operations.
- As the size of the transistors decreases, their speed can be increased, and the overall speed of the integrated chip will be increased (i.e., increasing clock rate).

Moore's Law

- Moore's Law:
 - Common understanding: computing power tends to approximately double every two years.
 - Actually: the number of transistors per square area will roughly double every two years.

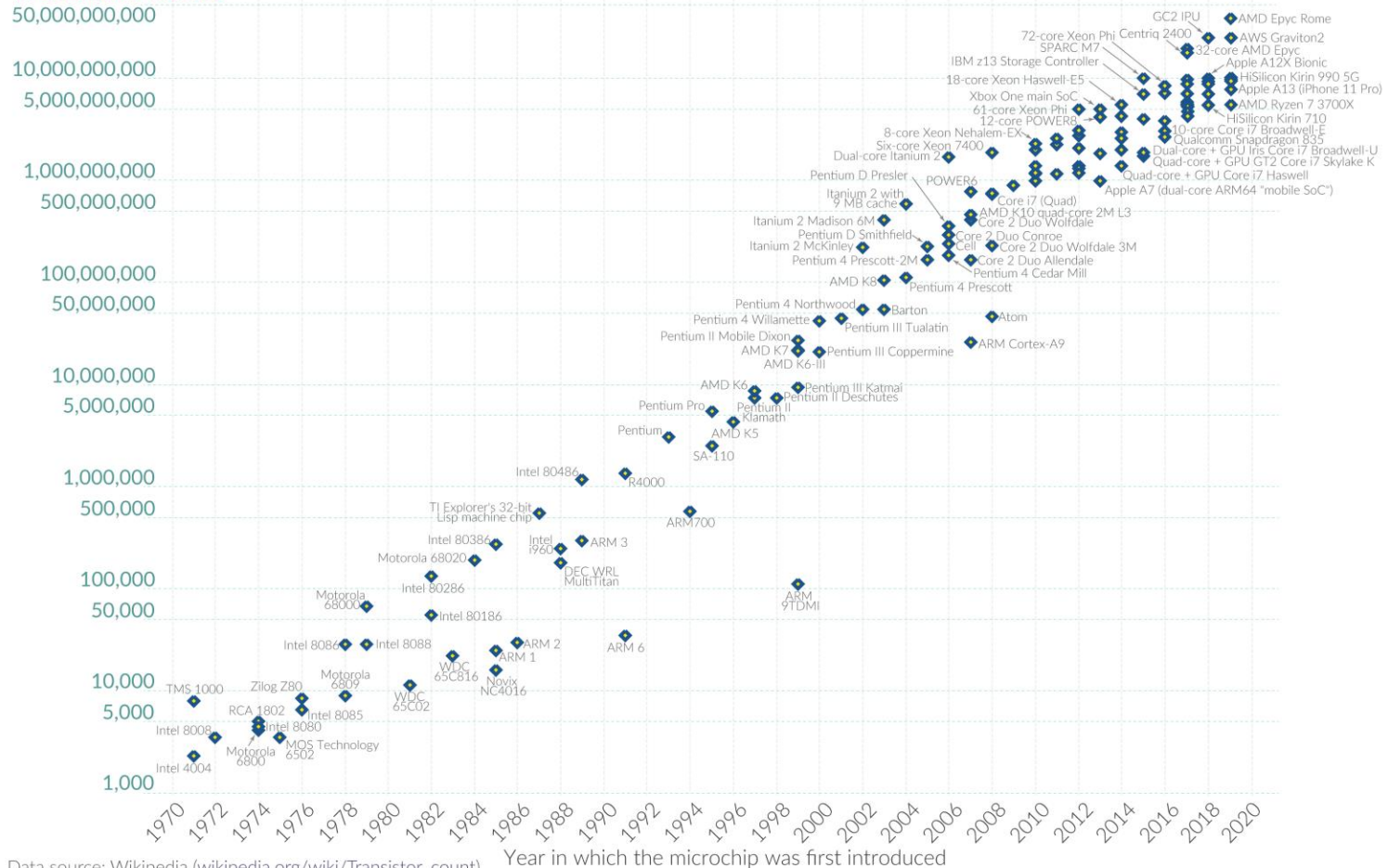
History of Moore's Law

Moore's Law: The number of transistors on microchips doubles every two years

Moore's law describes the empirical regularity that the number of transistors on integrated circuits doubles approximately every two years. This advancement is important for other aspects of technological progress in computing – such as processing speed or the price of computers.

Our World
in Data

Transistor count



Data source: Wikipedia (wikipedia.org/wiki/Transistor_count)

OurWorldinData.org – Research and data to make progress against the world's largest problems.

Licensed under CC-BY by the authors Hannah Ritchie and Max Roser.

What is the Problem?

- Hardware Trends have changed:
 - Transistors are no longer becoming faster.

Clock Speeds Have Stalled

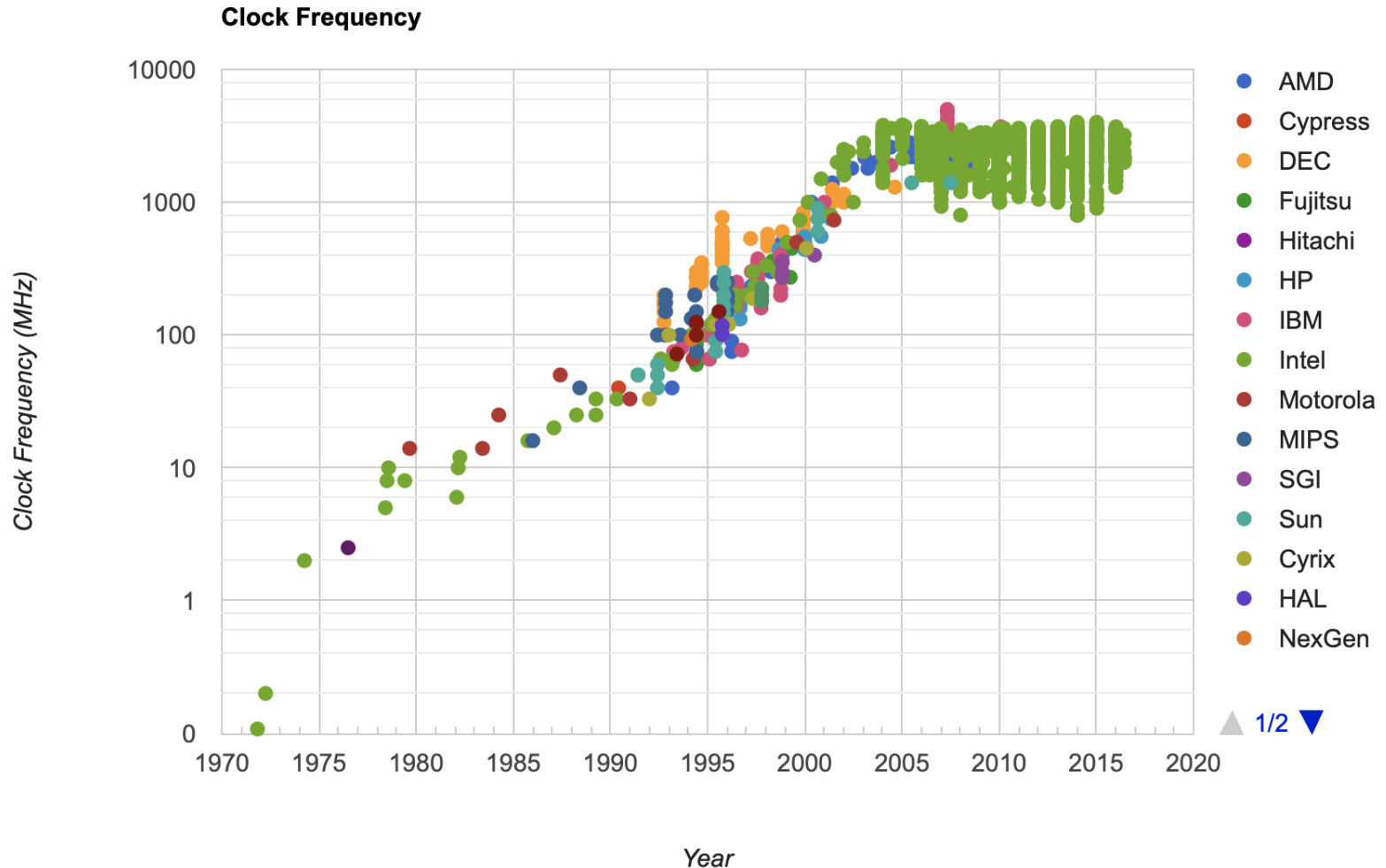


Diagram Source: http://cpudb.stanford.edu/visualize/clock_frequency

Why Did This Happen?

FIGURE 8

Power Density Over Time

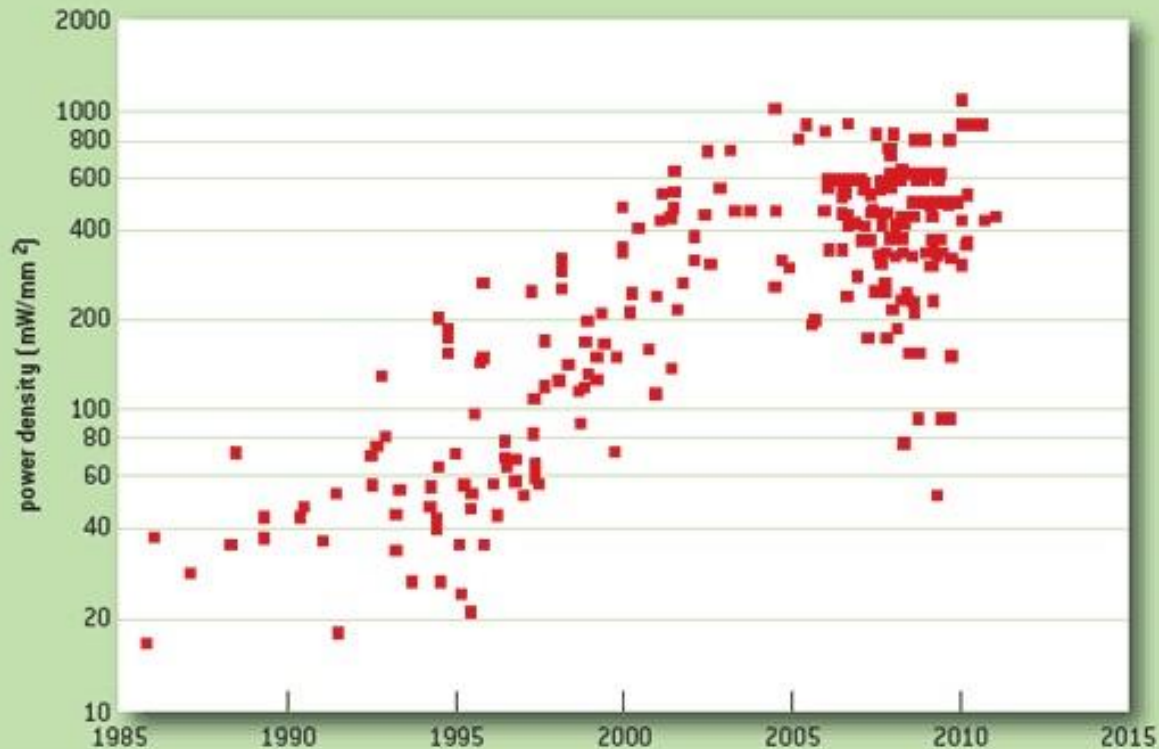


Diagram Source: <https://queue.acm.org/detail.cfm?id=2181798>

Why Did This Happen?

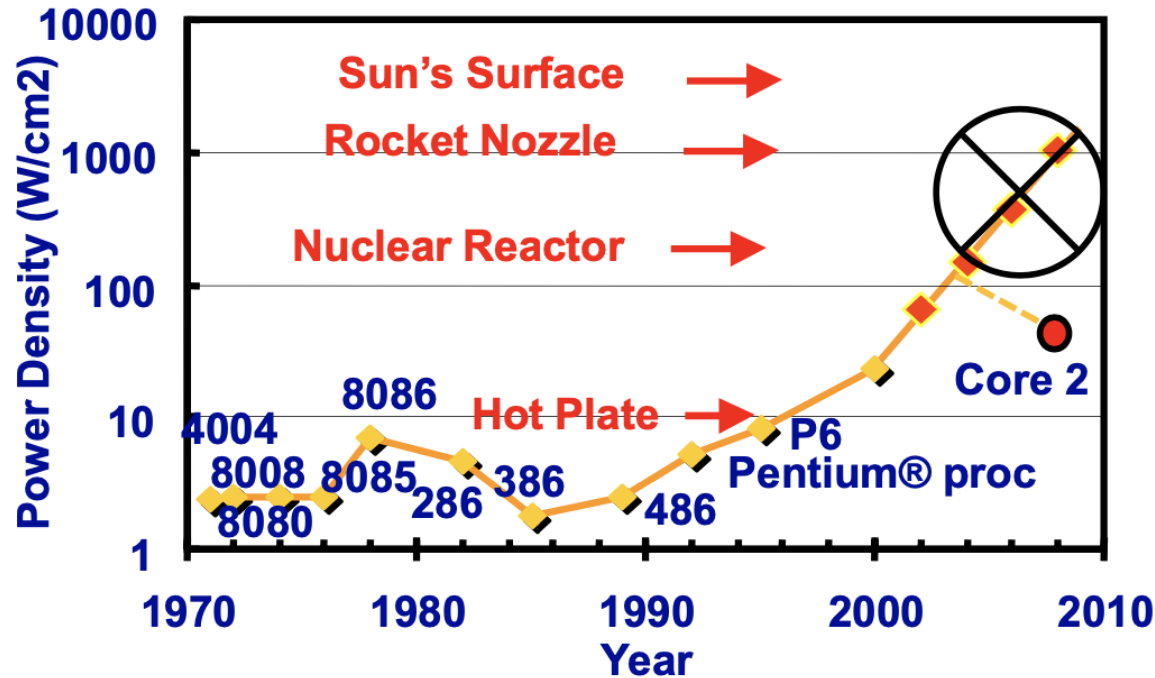
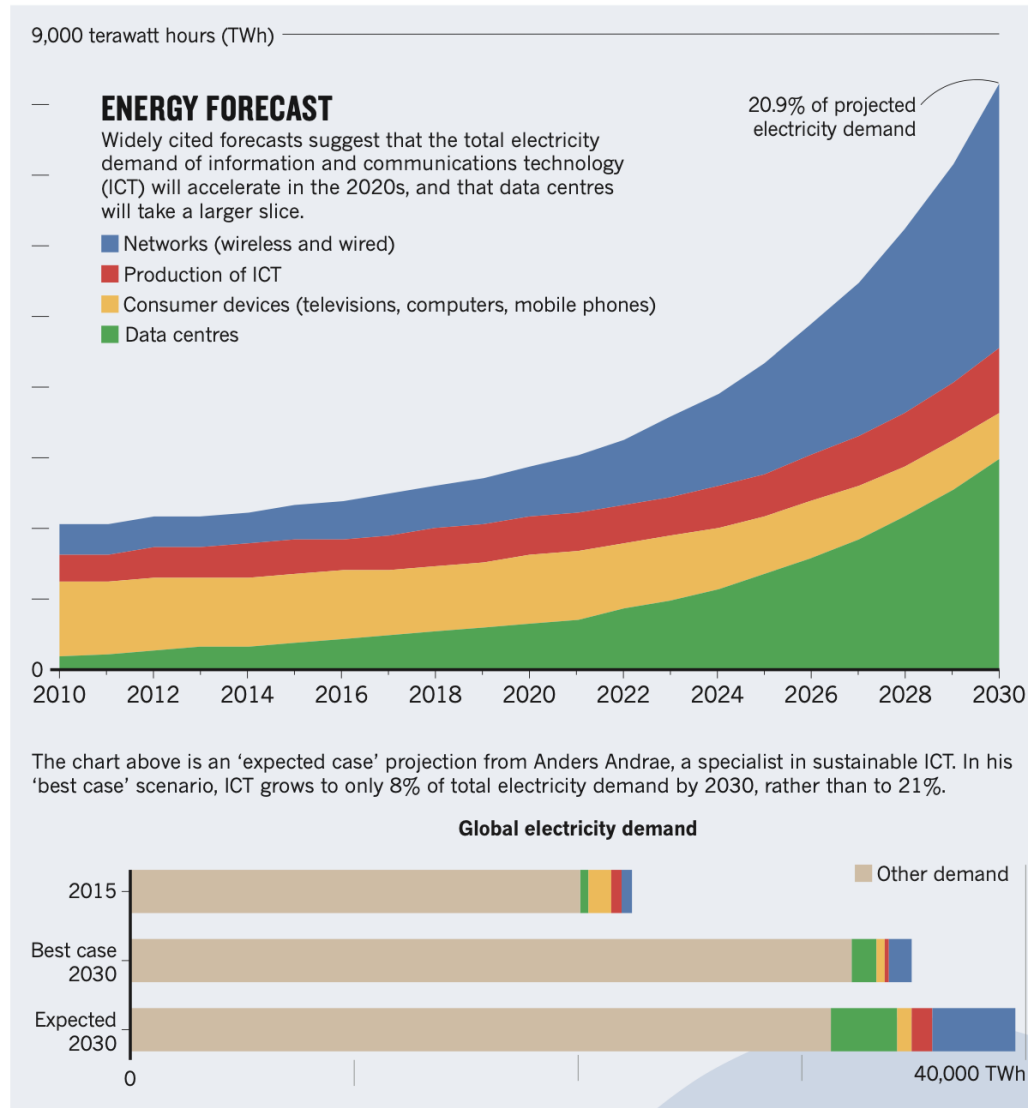


Figure 1.1 Today, typical CMOS microprocessors operate at around the power density of a nuclear reactor, *e.g.* 65 W and 130 W for Intel® Core™2 Duo E6320 (die size of 143 mm²) and Intel® Core™ i7-990X (die size of 239 mm²), respectively. Courtesy of Shekhar Y. Borkar, Intel Corp. [1].

Diagram Source: <https://escholarship.org/content/qt0616k5df/qt0616k5df.pdf>. "Germanium-Source Tunnel Field Effect Transistors for Ultra-Low Power Digital Logic", PhD thesis by Sung Hwan Kim, UC Berkeley

Why Does It Need to Happen

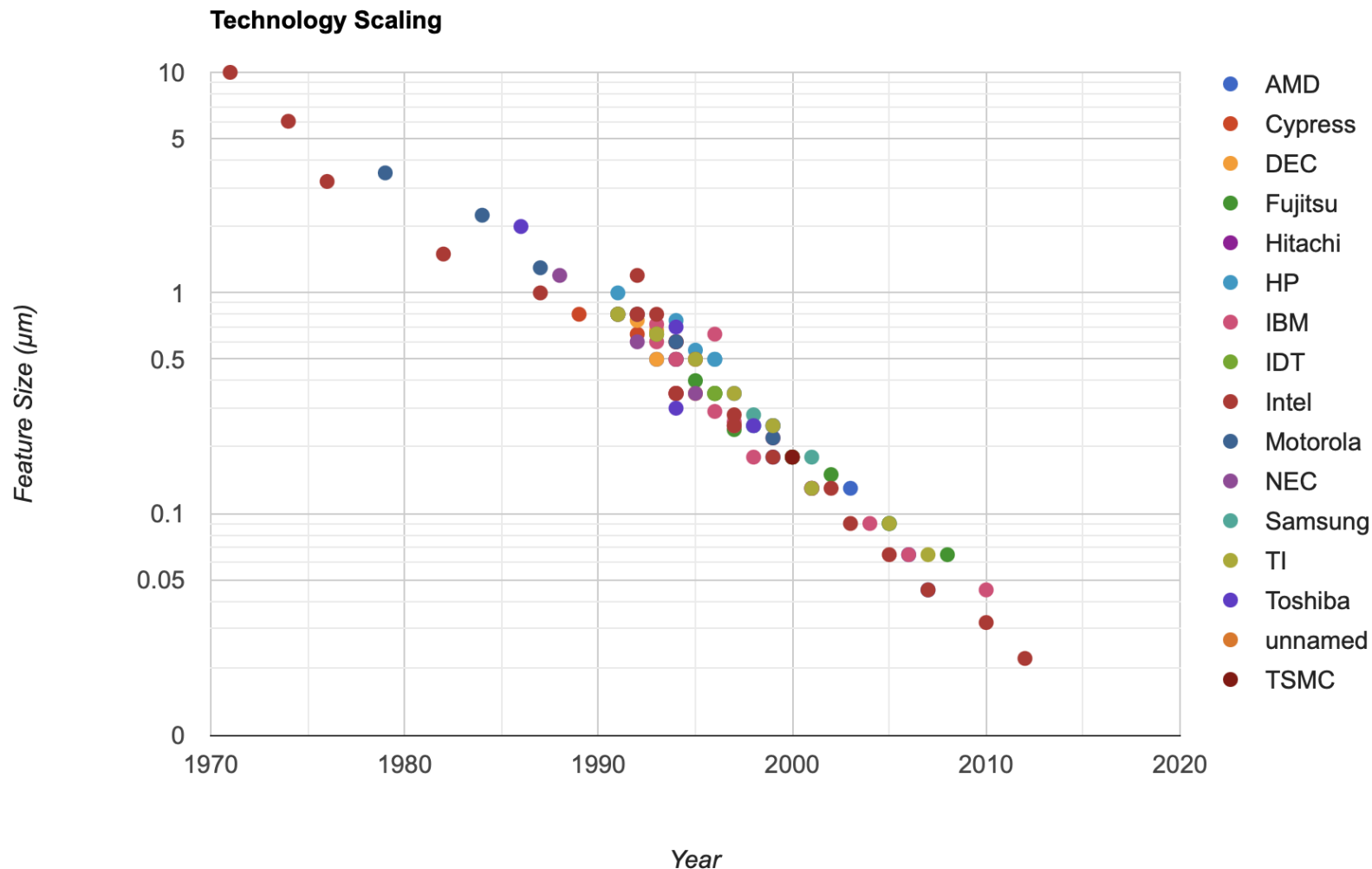


Source: <https://www.nature.com/articles/d41586-018-06610-y>

CPU power vs frequency

- The Power consumption of a CPU grows $\sim$ quadratic with frequency
- Doubling frequency $\Rightarrow$ 4x power consumption
- If we use the added transistors to do more at the same time, we can double throughput with only 2x power consumption.
- Or, we can lower the frequency and do even more at the same time, and reduce power consumption.

Can Moore's Law Continue?

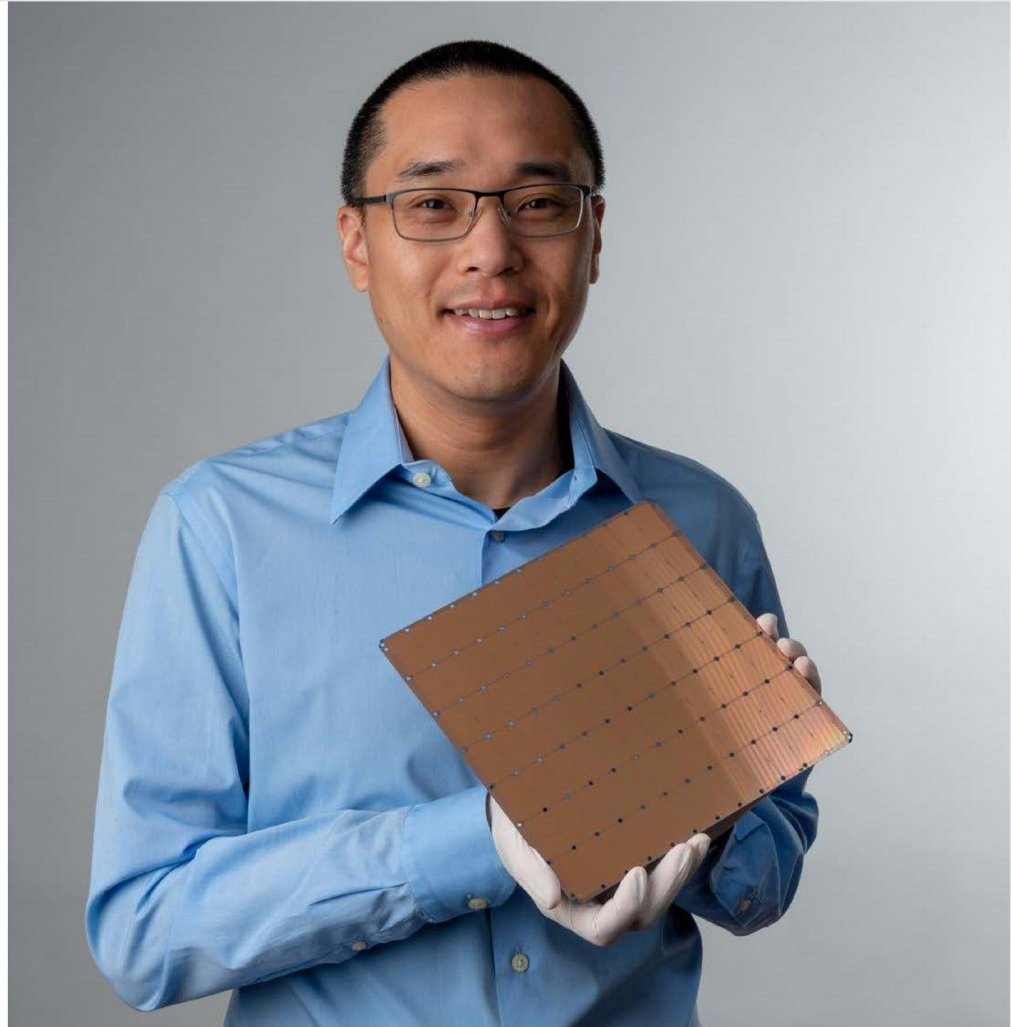


Source: http://cpudb.stanford.edu/visualize/technology_scaling_by_manufacturer

If We Can Not Go Small...

Largest Chip Ever Built

- 46,225 mm² silicon
- 1.2 trillion transistors
- 400,000 AI optimized cores
- 18 Gigabytes of On-chip Memory
- 9 PByte/s memory bandwidth
- 100 Pbit/s fabric bandwidth
- TSMC 16nm process



Source: <https://www.electronicshweekly.com/news/business/wafer-scale-back-2019-08/>

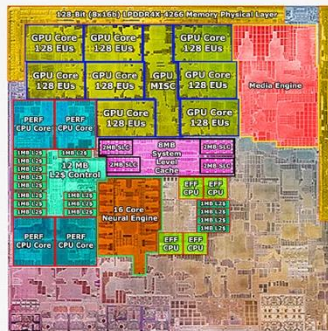
Real-World Multicore Architectures: Intel's Alder Lake-S Silicon

- Die shot of the Alder Lake- processors. These are Intel's newest (January 2022) processors used in the Core i9-12900K, Core i7-12700K, and Core i5-12600K, and other Intel chips.
 - 8-6 Performance CPU cores (normal CPU cores that are large, and run at high clock-speed)
 - 8-4 Efficient CPU cores (small CPU cores that run at reduced clock speed)



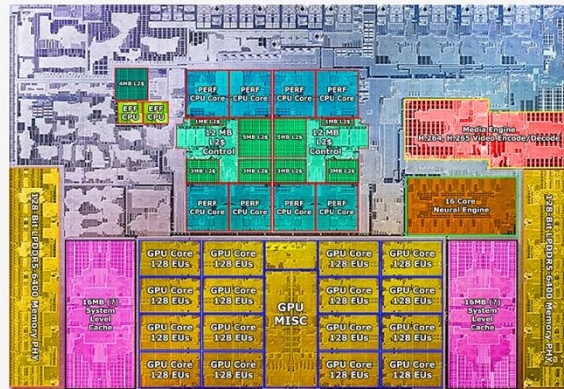
Slide by Lamont Samuels

Real-World Multicore Architectures: Apple Silicon



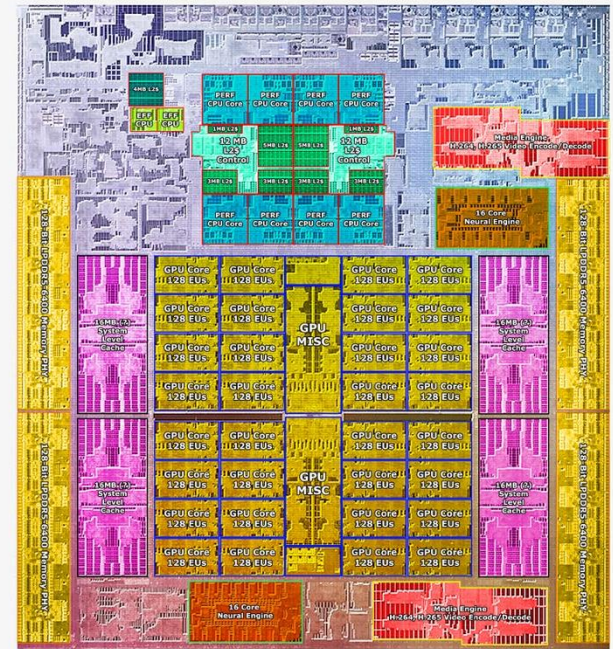
M1

- | CPU: 4x Performance Cores, ≤3.2 GHz + 4x Efficiency Cores, ≤2.06 GHz
- | GPU: 8x GPU Cores (1024 EUs)
24.576 „Threads-in-flight“
~1.27 GHz
2.6 FP32 TeraFLOPs
82 GigaTexel/s (64 TMUs)
41 GigaPixel/s (32 ROPs)
- | Memory: 128-bit LPDDR4X-4266
68.3 GB/s bandwidth



M1 Pro

- | CPU: 8x Performance Cores + 2x Efficiency Cores
- | GPU: 16x GPU Cores (2048 Execution Units)
49.152 „Threads-in-flight“
~1.27 GHz
5.2 FP32 TeraFLOPs
164 GigaTexel/s (128 TMUs)
82 GigaPixel/s (64 ROPs)
- | Memory: 256-bit LPDDR5-6400
204.8 GB/s bandwidth



M1 Max

- | CPU: 8x Performance Cores + 2x Efficiency Cores
- | GPU: 32x GPU Cores (4096 Execution Units)
98.304 „Threads-in-flight“
~1.27 GHz
10.4 FP32 TeraFLOPs
327 GigaTexel/s (256 TMUs)
164 GigaPixel/s (128 ROPs)
- | Memory: 512-bit LPDDR5-6400
409.6 GB/s bandwidth



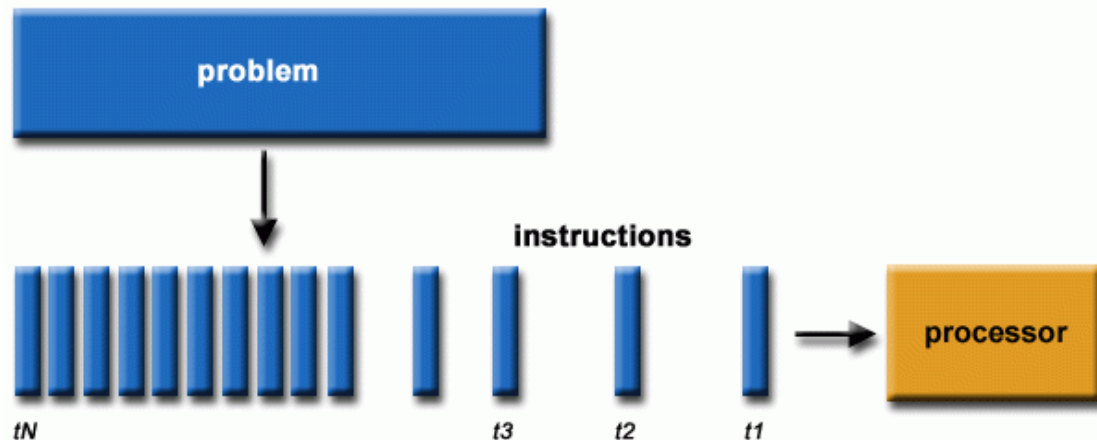
- 8x Zen2, ≤3.5 GHz
- 18x Cores (2304 EUs)
92.160 „Threads“
~2.23 GHz
10.28 FP32 TeraFLOPs
321 GTex/s (144 TMUs)
143 GPix/s (64 ROPs)
- 256-bit GDDR6-14000
448 GB/s bandwidth

Sources: M1 CPU clock frequency from Anandtech, chip images from Apple via Anandtech
Compiled and annotated by Locuza, October 2021

Slide by Lamont Samuels

Sequential Programs

- As programmers, we are overwhelmingly accustomed to developing sequential programs
 - A program solves a problem that is composed of a series of textual statements that specifies their order of execution.
 - Each statement is converted into an instruction that is executed by the CPU.
 - Sequential programs are deterministic(*), which means that every time the program runs the result is the same.

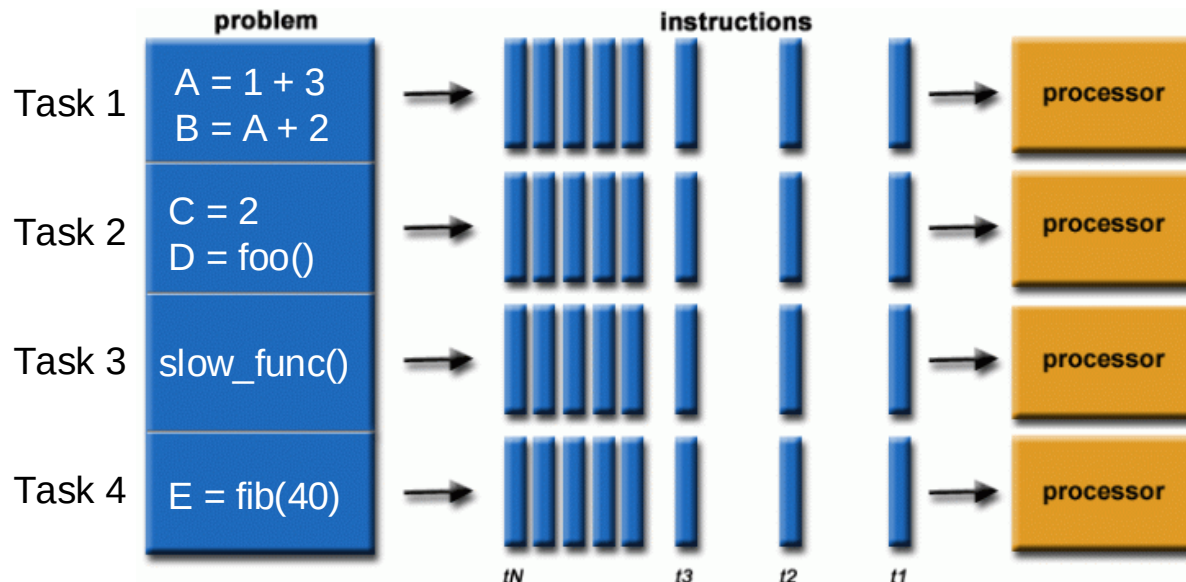


Slide by Lamont Samuels

Diagram Source: <https://hpc.llnl.gov/documentation/tutorials/introduction-parallel-computing-tutorial>

Programming on Parallel Architectures

- Programming on parallel computers requires programmers to break the problem into discrete parts (i.e., **tasks**) where each part can be ran on a separate core.
 - A task is a unit of work, where in this class, work is a series of program statements. Tasks can potentially to be executed in parallel but this is not a requirement.
 - At a high-level, a parallel program solves a problem that consists of multiple tasks running on multiple processors simultaneously .



Slide by Lamont Samuels

Diagram Source: <https://hpc.llnl.gov/documentation/tutorials/introduction-parallel-computing-tutorial>

Distributed / Shared Memory Systems

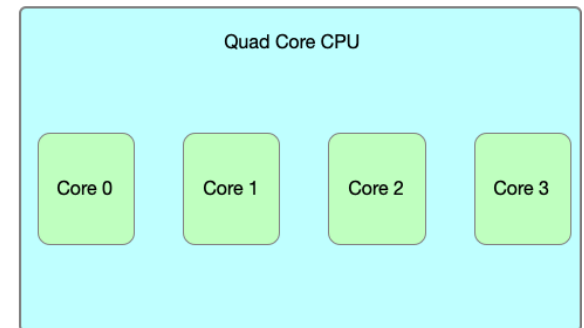
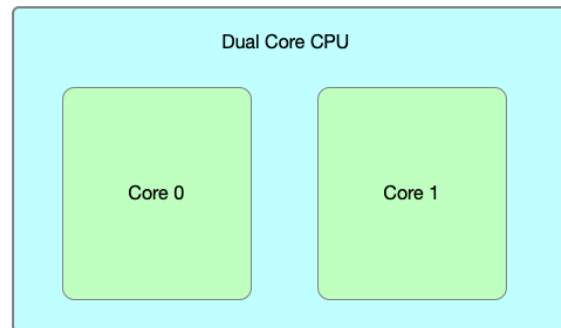
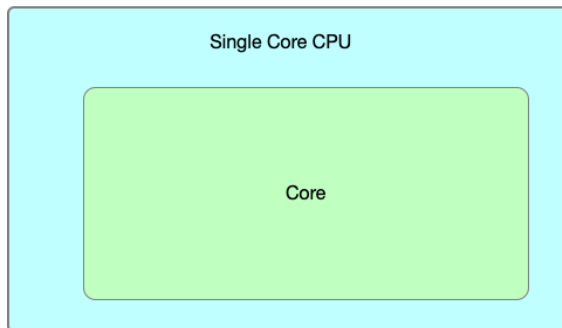
- Distributed Memory Systems:
 - Processors have their own memory
 - Explicit data transfer between processors
- Shared Memory Systems:
 - All processors access the same memory
 - No need for explicit data transfer

Distributed / Shared Memory Parallelism

- Distributed Memory Parallelism:
 - Tasks manage their own memory
 - Explicit data transfer between tasks
- Shared Memory Parallelism:
 - Tasks access shared memory
 - No need for explicit data transfer

Some CPU terminology

- **Core** (also known as **processor**) : synonymous with central processing unit (CPU).
- **Multicore** (also known as **multiprocessor**): more than one core on an integrated circuit.



Example: distributed memory system

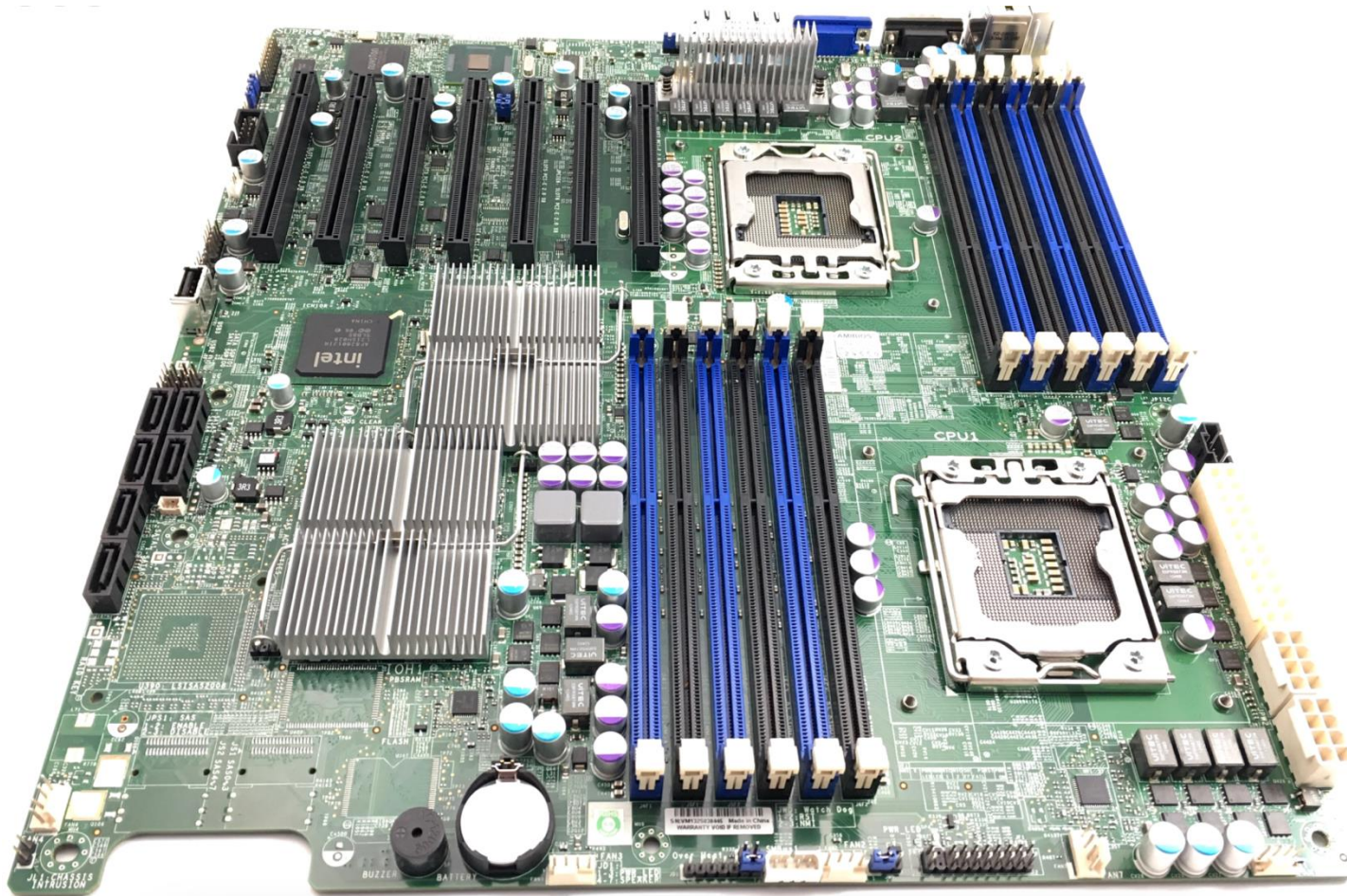


Aurora. Photo by Argonne Leadership Computing Facility

Shared Memory: Illusion vs Reality

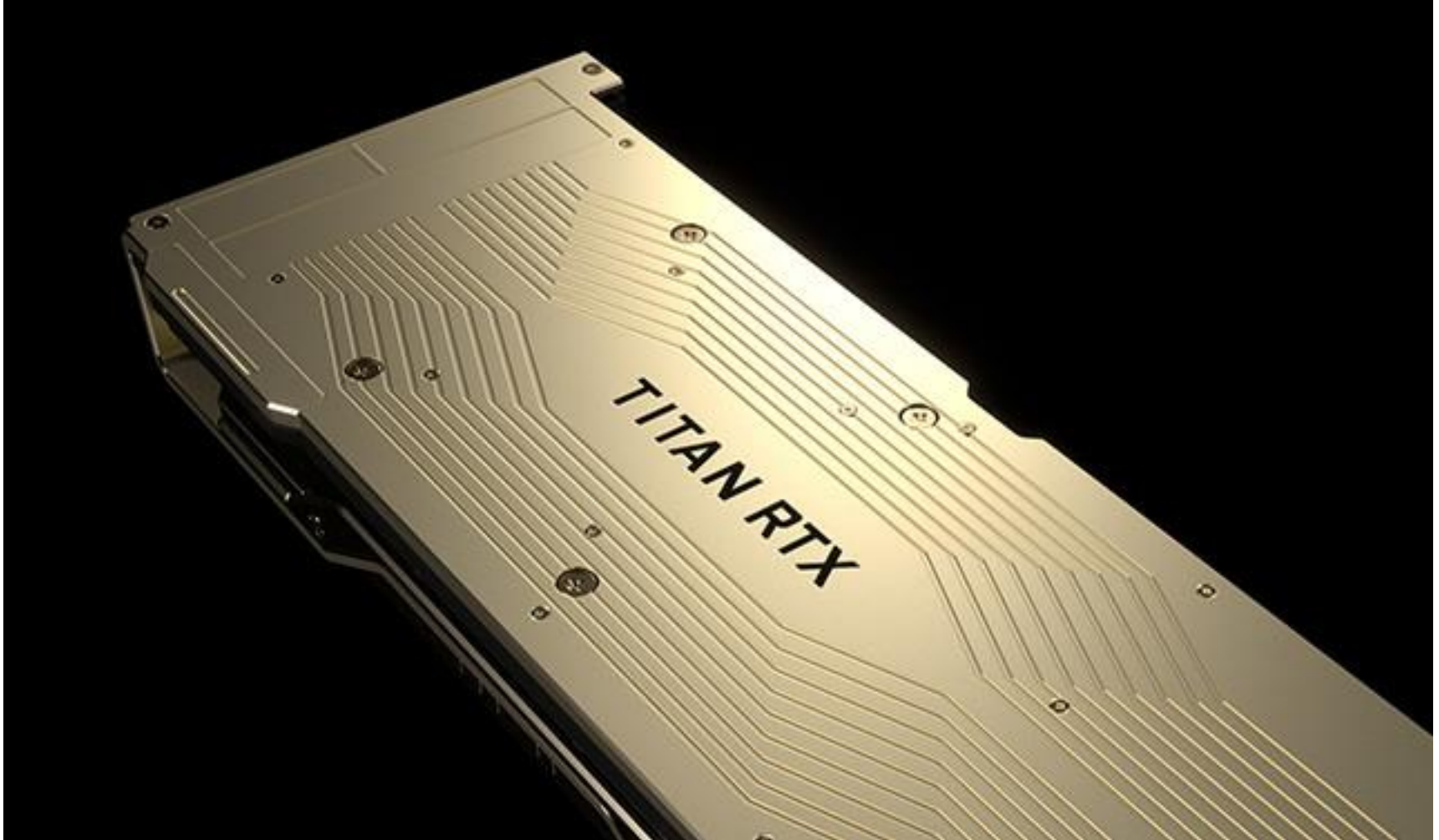
- Shared vs distributed is a question of programming model and hardware
 - Distributed memory parallelism on distributed memory systems (e.g. MPI on a cluster)
 - Shared memory parallelism on shared memory systems (e.g. Go, OpenMP, PThreads on a laptop)
 - Shared-memory programming on distributed-memory systems (e.g. PGAS)
 - Distributed-memory programming on shared-memory systems (e.g. MPI on a laptop)

Example: shared memory system?



<https://parts.metservers.com/mbd-x8dth-if-b-supermicro-xeon-eatx-dual-socket-lga1366-system-board>

Example: shared memory system?



<https://www.nvidia.com/content/dam/en-zz/Solutions/titan/titan-rtx/nvidia-titan-rtx-gallery-d-641-d.jpg>

Example: distributed memory system



Aurora. Photo by Argonne Leadership Computing Facility

Shared Memory Systems

The focus of this course will be on implementing parallel programs on **shared-memory** systems

- A shared-memory system consists of at least one multi-core CPU that allows all processors to access memory as a single global address space (also known as global memory).
 - Many of the architectures designed by Intel, AMD, ARM, and etc.
 - Architectures in many modern personal computers and phone.
- Shared memory can be implemented as a set of memory modules (e.g., caches, physical memory, etc.)

Symmetric Multiprocessing

- For this course, we focus on a specific type of shared memory system known as **uniform memory access (UMA)** systems
 - UMA systems provide each core with equal access and access times to global memory
- Specifically, we look at **symmetric multiprocessing (SMP)** systems that implement a UMA architecture
 - Each SMP processor is identical
 - All processors can access the shared main memory at the same speed.
 - Controlled by a single operating system instance that treats all processors equally (no processor is reserved for a specific purpose).
 - SMP systems are tightly coupled multiprocessor systems,
 - All processors can execute different programs (and with different data) in parallel.
 - All processors share common resources (e.g., memory, I/O device, interrupt system) all on the same system bus.

Concurrency vs. Parallelism

- You may hear the terms **concurrent** and **parallel** referring to the same notion of executing a task at the same time; however, there is a key distinction between the two.
 - Concurrency is about dealing with tasks that are logically happening simultaneously. Two tasks are concurrent if they can be logically active at some point in time.
 - Parallelism is about dealing with tasks that are physically happening simultaneously.
- Concurrency provides a way to structure a solution to a problem that may (but not necessary) have tasks executed in parallel.

Slide by Lamont Samuels

Parallel Scalability

- If we break up a program into n tasks and run them in parallel, we hope for n times speedup
- The reality often looks different

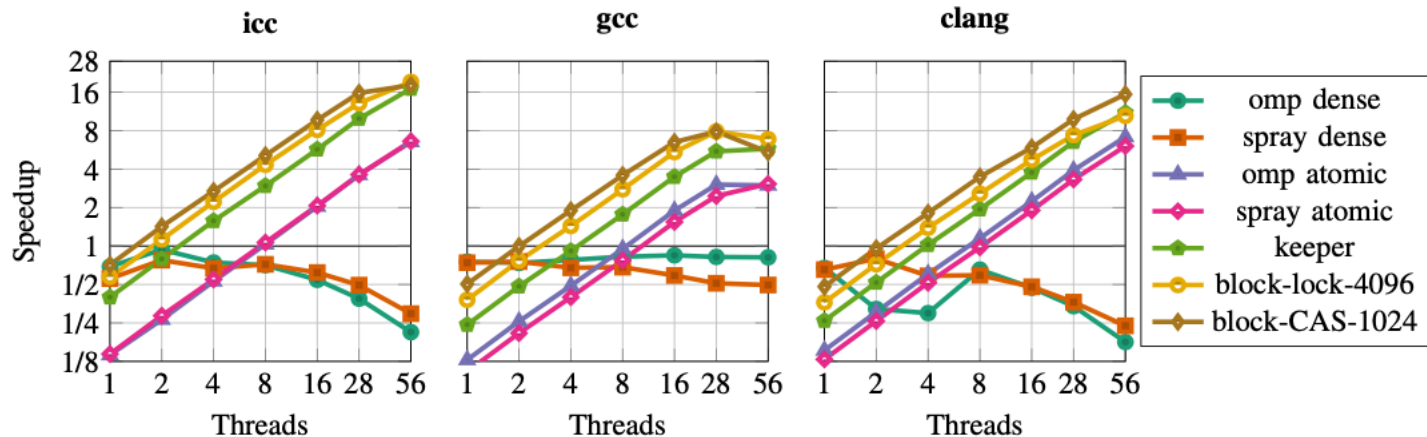


Fig. 11. Speedup of built-in OPENMP and selected SPRAY reduction implementations over the sequential convolution back-propagation, on Intel, GNU, and LLVM compilers. All parallel implementations have some overhead. Hence, their lines start below 1, indicating a slowdown, and rise above 1, indicating a speedup as more threads are added. The sequential version is created with each compiler; hence the baseline performance differs between subplots, and better scalability with one compiler does not imply better absolute run time.

<https://ieeexplore.ieee.org/document/9460492>

Parallelism Is Here To Stay

- We can not predict the future, but we can make educated guesses.
- Unless there is some radical break in paradigms, computers will continue to be increasingly parallel
- It is also possible that they will become increasingly heterogeneous

What will you learn in this class?

- Main objectives:
 - Best practices for designing and implementing parallel programs.
 - Synchronization mechanisms to maintain deterministic results
 - Schemes for processing tasks in parallel: algorithms, patterns and techniques to help with maintaining performance when scaling your application.
 - Understanding parallel architectures for fine tuning performance
- **Overall Goal:** Regardless of your programming language of choice (C, Java, C++, Python, etc.), you should be able to use the techniques, algorithms, patterns, and practices taught in this class and apply them to developing parallel applications in those languages.

Slide by Lamont Samuels

Go Bootcamp

Design of Go

Go was designed by Google (specifically Robert Griesemer, Rob Pike, and Kenneth Thompson in 2007) to solve problems that Google faces.

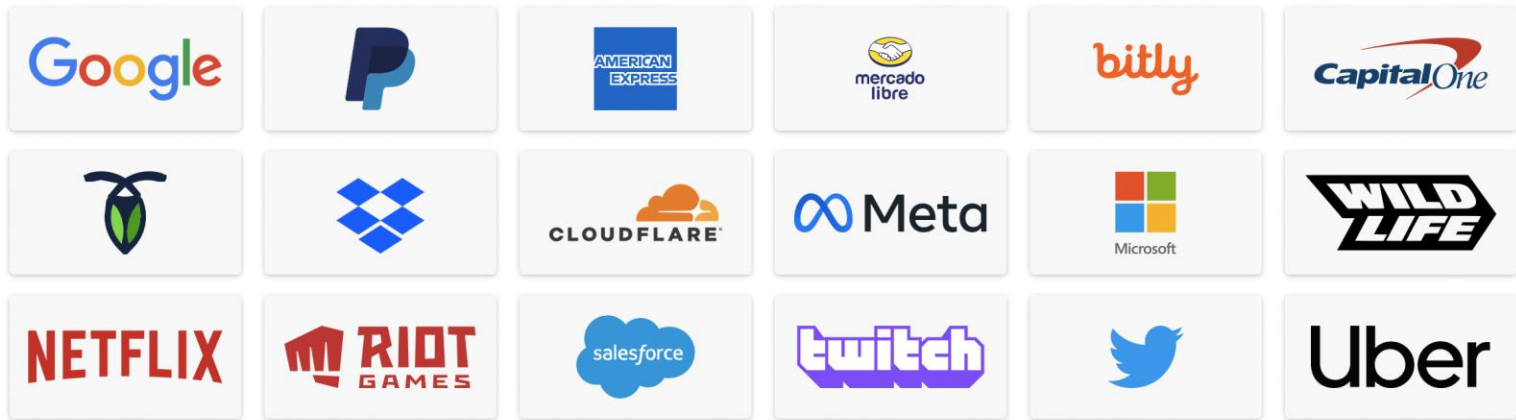
- Goals
 - Eliminate slowness
 - Inefficiencies
 - Maintain and improve scale
 - Make a concurrent language
- Types of problems Google faces?
 - C++ for servers, plus lots of Java and Python mixed in
 - Large employee base
 - Millions upon Millions of lines of code for various projects
 - Distributed build system
 - Millions of compute cluster machines, needing to perform tasks

Slide by Lamont Samuels

Who uses Go?

Companies using Go

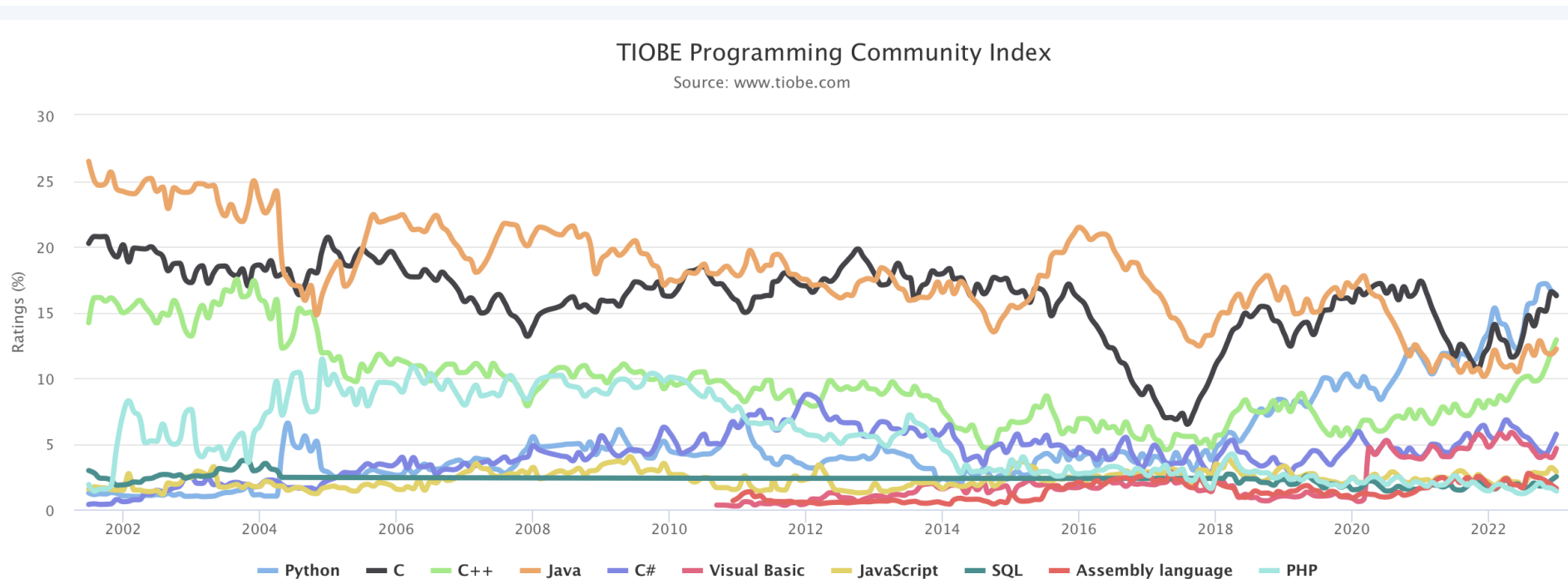
Organizations in every industry use Go to power their software and services [View all stories](#)



- Diagram Source: <https://go.dev>

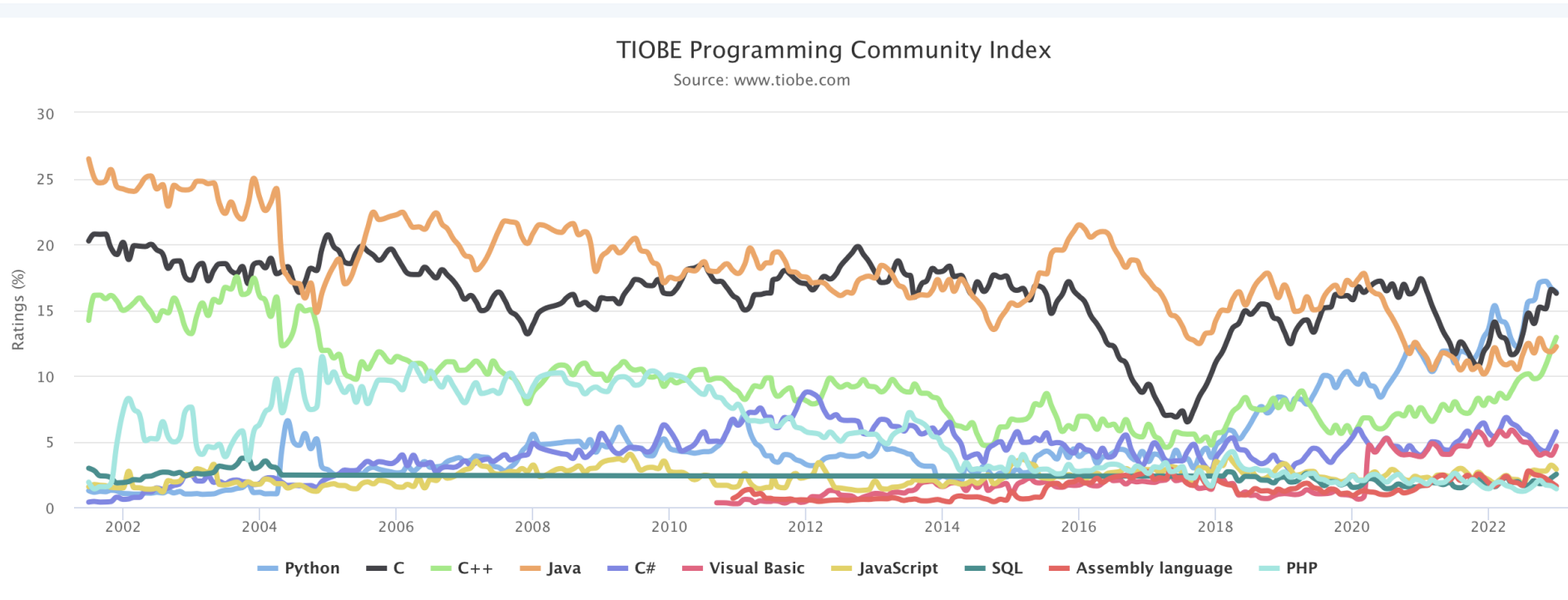
Slide by Lamont Samuels

Who does not use Go?



Almost everyone else.

Who does not use Go?



Almost everyone else. (Including your instructor).

Why are we using Go?

- Go is relatively easy to learn
- Go has concurrency built-in
 - (unlike many other languages)
- Go has a very general way of expressing parallelism, and can teach us most concepts that can be found elsewhere

How are we going to learn Go?

- We are actually going to learn Go by going through examples shown on: <https://gobyexample.com>
- Constructs you should know:
 1. Values
 2. Variables
 3. Constants
 4. Importing
 5. Iteration statement: For, Range
 6. Selection statements: If/Else, Switch
 7. Core Data Structures: Arrays, Slices, Maps
 8. Functions
 9. Multiple Return Values
 10. Variadic Functions
 11. Pointers (VERY IMPORTANT!)
 12. Structs
 13. String Formatting and Functions
 14. Generics

Slide by Lamont Samuels